Project Title

2D Chess Game Using C and SDL3 (Two-Player Chess Game)

Group Members (maximum 3 members per group)

- Syeda Huda Asim (Roll No. 25K-6540)
- Syeda Eman Zehra (Roll No. 25K-0811)
- Aliza Syed (Roll No. 25K-6580)

Submission Date

26 april 2026

---

1. Executive Summary

Overview:

This project presents the development of a fully functional 2D Chess Game using the C programming language and the SDL3 graphics library. The system displays an 8×8 chessboard, loads graphical textures for chess pieces, and allows two players to play by following standard movement rules. The project demonstrates key programming fundamentals such as arrays, enums, loops, conditionals, and event-driven programming.

Key Findings:

- Successfully implemented a graphical chessboard with interactive gameplay.
- Achieved correct movement validation for all chess pieces.
- Integrated SDL3 with C for rendering and user interaction.
- Developed a turn-based two-player system with capture functionality.

---

2. Introduction

Background:

Chess is a strategic board game that involves logical thinking and rule-based movement. Implementing a chess game in programming helps strengthen problem-solving skills and understanding of algorithms. This project is relevant to Programming Fundamentals as it applies core concepts like arrays, conditionals, loops, and event handling in a real-world application.

Project Objectives:

- To implement a fully functional 8×8 chessboard using SDL3.
- To load and display chess piece textures.
- To detect mouse clicks and convert them into board positions.
- To validate legal movements for all chess pieces.
- To demonstrate the use of arrays, enums, and structured programming concepts.

---

3. Project Description

Scope:

The project includes:

- Graphical rendering of a chessboard
- Display of all chess pieces in their initial positions
- Mouse-based interaction for selecting and moving pieces
- Movement validation for all standard chess pieces
- Turn-based gameplay between two players

Excluded Features:

- Check and checkmate detection
- Special moves such as castling, en-passant, and pawn promotion
- Artificial Intelligence (AI) opponent

Technical Overview:

- **Programming Language:** C
- **Graphics Library:** SDL3
- **Development Tools:** Code::Blocks, Dev C++, GCC
- **Assets:** PNG images for chess pieces

---

4. Methodology

Approach:

The project was developed using a structured programming approach. The following steps were followed:

1. Initialize SDL3 video subsystem.

2. Create a window and renderer.
3. Load PNG textures for all chess pieces.
4. Draw an 8×8 chessboard.
5. Handle user input through mouse clicks.
6. Validate selected piece and its movement.
7. Update board state if the move is valid.
8. Re-render the screen continuously (~60 FPS).
9. Exit on quit event.

Roles and Responsibilities:

- **Syeda Huda Asim:**
  - Implemented core game logic and movement validation
  - Designed board structure using arrays and enums
  - Managed overall program structure
- **Syeda Eman Zehra:**
  - Worked on SDL3 graphics rendering and texture handling
  - Implemented mouse event handling and user interaction
- **Aliza Syed:**
  - Assisted in testing and debugging
  - Contributed to documentation and report writing
  - Verified functionality and handled edge case testing

---

5. Project Implementation

Design and Structure:

The chessboard is represented using a 2D array, where each element stores information about a specific piece. Enums are used to define different piece types, making the program more organized and readable. SDL3 is used to render the board and textures, while event handling manages user interaction.

Functionalities Developed:

- Rendering of an 8×8 chessboard with alternating tile colors
- Loading and displaying chess piece textures
- Mouse click detection for selecting and moving pieces
- Movement validation for:
  - Pawn
  - Rook
  - Bishop
  - Knight
  - Queen

    - King
- Turn-based gameplay (white and black)
- Capture system and game-ending condition when a king is captured

Challenges Faced:

- **Complex Movement Logic:** Implementing different rules for each piece required careful condition handling.
- **Mouse Input Handling:** Mapping screen coordinates to board positions was challenging.
- **Graphics Integration:** Properly loading and rendering textures in SDL3 required debugging.

Solutions:

- Broke movement logic into smaller conditions for each piece
- Used coordinate calculations to map mouse clicks accurately
- Referred to SDL3 documentation and tutorials to fix rendering issues

---

6. Results

Project Outcomes:

The project successfully achieved its main objectives. The game runs smoothly, displays the chessboard correctly, and enforces valid movement rules for all pieces.

Screenshots and Illustrations:

Example for chess board

```c
void drawChessboard(SDL_Renderer* renderer, SDL_Texture** pieceTextures) {
        SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
        SDL_RenderClear(renderer);

    for (int row = 0; row < 8; row++) {
        for (int col = 0; col < 8; col++) {
            SDL_FRect tile = { col * TILE_SIZE , row * TILE_SIZE, TILE_SIZE, TILE_SIZE ]
            if ((row + col) % 2 == 0)
                SDL_SetRenderDrawColor(renderer, 240, 217, 181, 255);
            else
                SDL_SetRenderDrawColor(renderer, 181, 136, 99, 255);

            SDL_RenderFillRect(renderer, &tile);

            PieceType piece = board[row][col];
            if (piece != EMPTY && pieceTextures[piece])
                SDL_RenderTexture(renderer, pieceTextures[piece], NULL, &tile);
        }
    }

    SDL_RenderPresent(renderer);
}
```

Game ending and piece location changing

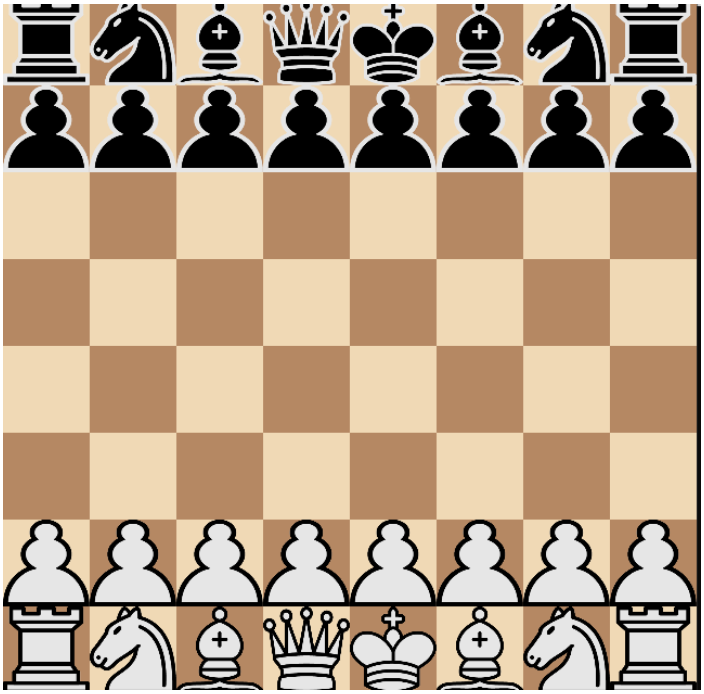
```c

                if (validMove) {
                    board[row][col] = selPiece;
                    board[selectedRow][selectedCol] = EMPTY;
                    whiteTurn = !whiteTurn;

                    // Check if either king is captured
                    int whiteKingExists = 1, blackKingExists = 1;
                    for (int r = 0; r < 8; r++) {
                       for (int c = 0; c < 8; c++) {
                          if (board[r][c] == WHITE_KING) whiteKingExists = 0;
                          if (board[r][c] == BLACK_KING) blackKingExists = 0;
                       }
                    }
                     if (whiteKingExists || blackKingExists) {
                     printf("Game Ended\n");
                     running = 0;  // Exit main loop
                     }

                }

```



Testing and Validation:

The following test cases were performed:

- Pawn forward movement – PASS
- Knight L-shape movement – PASS
- Rook straight movement – PASS
- Invalid rook diagonal move – PASS
- Capture logic – PASS

All tested scenarios produced correct and expected results.

7. Conclusion

Summary of Findings:

This project demonstrates a strong understanding of programming fundamentals and successful implementation of a graphical application using C and SDL3. It effectively converts theoretical programming concepts into a working game.

Final Remarks:

The project can be further improved by implementing advanced chess rules such as checkmate detection, special moves, and an AI opponent. These enhancements would make the game more complete and closer to real-world chess applications.